

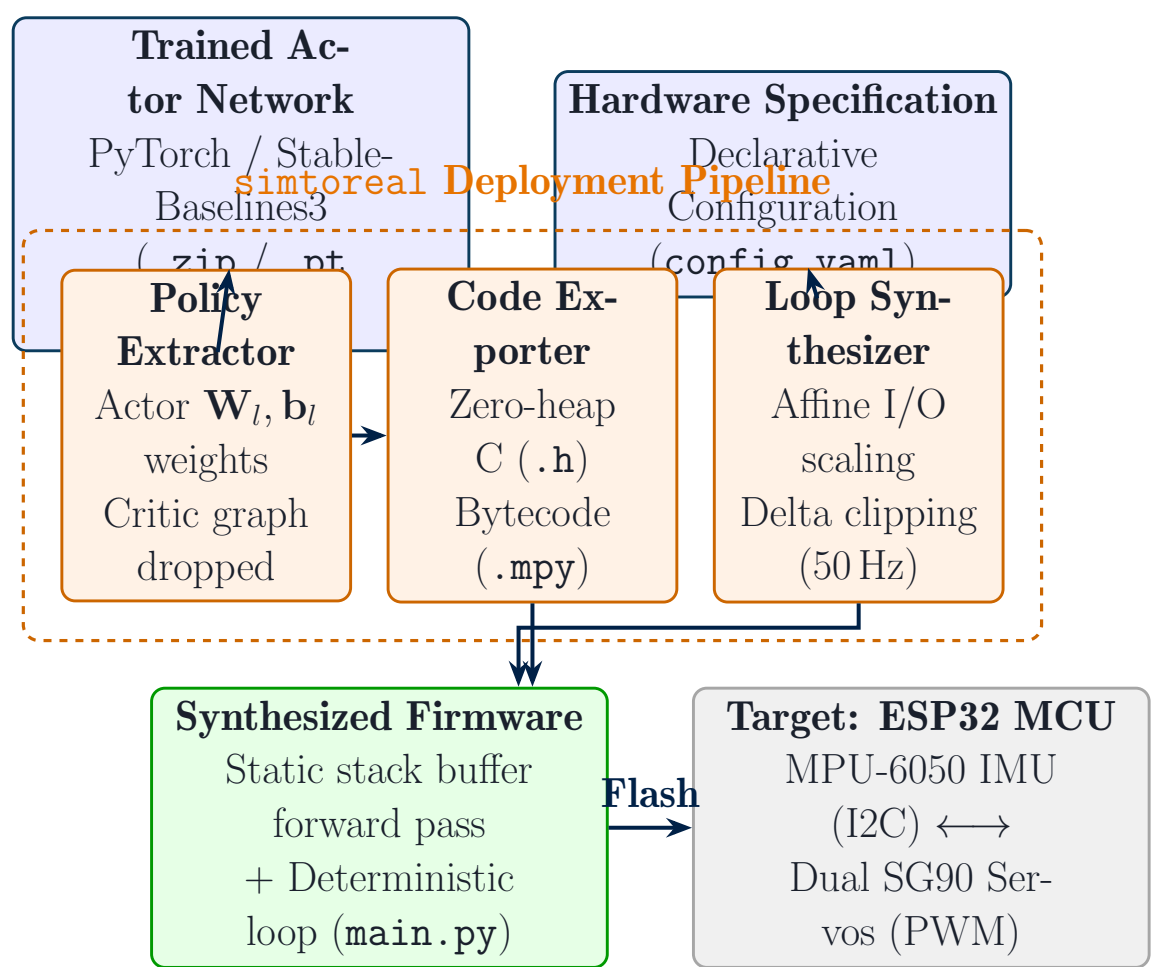
1. Problem Scenario & Motivation

The Scenario: While reinforcement learning (RL) policies achieve dynamic agility in high-fidelity physics simulations, transferring them to ultra-low-cost, resource-constrained microcontrollers (<520 kB RAM, bare-metal) requires tedious manual weight translation, custom forward passes, and error-prone peripheral driver programming.

Key Objectives & Proposed Framework:

- **Dual-Target Export:** Compile PyTorch / Stable-Baselines3 actors directly into zero-allocation C headers or precompiled MicroPython bytecode (.mpy).
- **Declarative Loop Synthesis:** Generate complete 50 Hz control firmware (main.py / C loop) from a single config.yaml specifying I2C IMU pins, normalization scales, and safety delta clipping.
- **Guaranteed Determinism:** Dual static stack buffers eliminate runtime heap allocations (malloc), preventing memory fragmentation and latency spikes.

2. System Architecture & Pipeline

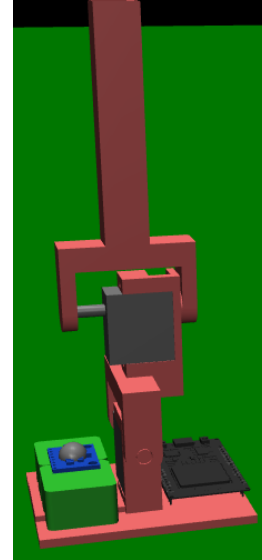


Pipeline Flow: (1) Extract weights → (2) Export zero-heap C / bytecode → (3) Synthesize 50 Hz I2C/PWM control loop.

3. Literature Review & Framework Comparison

Framework	Target ware	Hard- ware	RAM Us- age	Python Stack	RL Key Limitation / Feature
ROS 2 / Orbit	Linux x86 (RPi 4, Jetson)	SBCs	>100 MB	Native Torch	SB3 / High OS overhead; cannot run bare-metal microcontrollers.
TFLM	Edge MCUs (Cortex-M, ESP32)		20-50 kB	TensorFlow only (No direct SB3)	General tensor interpreter runtime adds overhead for compact MLPs.
MCUNet	Microcontrollers (Cortex-M7)		30-100 kB	PyTorch Vision	Tailored for visual CNNs; no continuous RL control loops.
RLtools	Microcontrollers & Bare-metal		<10 kB	Custom only (No Torch/SB3)	C++ Requires training inside proprietary C++ framework; no standard Py-Torch.
simtoreal (Ours)	Bare-Metal MCUs (ESP32, RP2040)		<19 kB Flash 0 Heap (C)	Direct SB3 & PyTorch Im- port	Automated peripheral drivers, zero-heap C headers, declarative YAML.

4. Experimental Testbed: Robo1



(a) MuJoCo Simulation Model



(b) 3D-Printed Physical Robo1

System Specifications & Training Setup:

- **Kinematics & Mass:** 2-DOF dynamic self-righting robot ($M = 94.8$ g, $H = 145$ mm). Joint 1 (Roll) + Joint 2 (Pitch) actuated by SG90 micro-servos.
- **Embedded Core:** ESP32 dual-core MCU, MPU-6050 6-axis IMU over I2C (400 kHz).
- **Control Policy:** 5-64-64-2 tanh MLP actor (4,674 parameters, 18.7 kB).
- **Observation & Action:** $\mathbf{o}_t = [\phi_{\text{roll}}, \theta_{\text{pitch}}, a_z, q_1, q_2]$, Action $\Delta q \in [-0.08, +0.08]$ rad at 50 Hz.
- **Domain Randomization:** Body mass ($\pm 15\%$), damping ($\pm 20\%$), motor gain ($\pm 15\%$), sensor noise ($\sigma = 0.015$ rad).

5. Experimental Results & Performance Benchmarks

Evaluation Domain	Evaluated Target	Measured Metric & Empirical Findings
Host Numerical Parity ($n = 4,609$)	MicroPython Target	Max Abs Error: 1.2×10^{-6} (100% sign match)
Host Numerical Parity ($n = 4,609$)	C Header Target	Max Abs Error: 2.9×10^{-6} (100% sign match)
MuJoCo Righting	Nominal C Header troller	Con- 97.2% Success (100% standard falls in 1.15 s mean)
MuJoCo Rand Physics	Domain- C Header troller	Con- 97.2% Success (1.38 s mean get-up time, high robustness)
ESP32 Flash footprint	Flash Foot- print	Compiled Binary 75.1 kB (fits standard 4 MB Flash easily)
ESP32 RAM	MicroPython Heap	MicroPython 119.6 kB (72% of free heap; avoids heap thrashing)
Hardware I/O Budget (50 Hz)	I2C IMU + PWM	4.2 ms (Only 21% of 20 ms real-time deadline)
Inference Profile	Latency MicroPython	155.2 $\pm$ 7.4 ms → Compiled C header required for 50 Hz

Benchmark Findings:

- **Numerical Equivalence:** Single-precision float parity confirmed with max error $< 3 \times 10^{-6}$ against PyTorch.
- **Hardware I/O Deadlines:** Peripheral access takes only 4.2 ms (21% of 20 ms budget), proving real-time feasibility.
- **Zero-Heap C Header:** Eliminates interpreted bytecode latency, delivering deterministic 50 Hz closed-loop control.

6. Conclusion & References

Conclusion: The **simtoreal** framework bridges Python RL development and bare-metal microcontroller execution. Evaluations confirm numerical parity and 97.2% dynamic get-up success in MuJoCo, paving the way for real-world dynamic trials.

References: [1] Hwangbo et al., *Sci. Robot.* 2019. [2] Rudin et al., *CoRL* 2022. [3] Mittal et al., *IEEE RA-L* 2023. [4] David et al., *MLSys* 2021. [5] Lin et al., *NeurIPS* 2020. [6] Eschmann et al., *JMLR* 2024.